

Desarrollo Web Entorno Servidor

Unidad 5: Caso 1

Índice

1. [Introducción](#)
2. [Diseño e implementación del polimorfismo](#)
 - 2.1 [Clase base común](#)
3. [Integración en la interfaz de usuario](#)
4. [Pruebas y validación](#)
5. [Conclusión](#)
6. [Referencias](#)

1. Introducción

En esta unidad se ha trabajado la aplicación práctica del polimorfismo dentro del proyecto de la tienda online. El objetivo no ha sido únicamente demostrar el concepto a nivel teórico, sino integrarlo en una funcionalidad real del sistema, concretamente en la lógica de búsqueda y ordenación de productos.

Para ello, se han desarrollado dos clases que comparten la misma interfaz pública pero presentan comportamientos internos distintos. Esta diferencia permite aplicar el principio de polimorfismo de forma dinámica, manteniendo una estructura clara y reutilizable dentro de la capa de negocio.

Además, la implementación no se limita al backend, sino que se ha integrado con la interfaz de usuario desarrollada en *tienda.php*, permitiendo al usuario interactuar con la ordenación mediante la propia UI.

2. Diseño e implementación del polimorfismo

El polimorfismo se ha implementado mediante la creación de dos clases dentro de la capa de negocio: *Prod* y *ProdFiltro*. Ambas reciben una instancia del repositorio de datos y exponen el mismo método público *buscar(string \$term): array*.

Desde el punto de vista externo, ambas clases pueden utilizarse exactamente de la misma forma. Sin embargo, su comportamiento interno es distinto. La clase *Prod* realiza una búsqueda estándar sobre el repositorio, mientras que *ProdFiltro* amplía este comportamiento incorporando lógica de ordenación por precio en función del parámetro *orden* recibido mediante *\$_GET*.

De este modo, el mismo método (*buscar*) mantiene una interfaz común pero ejecuta lógicas diferentes dependiendo de la clase concreta que lo implemente, lo que constituye un ejemplo práctico de polimorfismo dinámico.

```
<?php
require_once __DIR__ . '/baseprod.php';

/**
 * Clase Prod
 * Implementa la lógica de negocio para la búsqueda de productos.
 */
class Prod extends BaseProd {

    /**
     * Busca productos aplicando una regla mínima:
     * si el término existe y tiene menos de 2 caracteres, devuelve un array vacío.
     */

    public function buscar(string $term=''): array {
```

```

        $term = trim($term ?? '');
        if ($term !== '' && mb_strlen($term) < 2) return [];
        return $this->repo->buscar($term);
    }
}

```

En la clase Prod se implementa el método *buscar(string \$term)*: array realizando una búsqueda directa sobre el repositorio de datos. Esta clase representa el comportamiento estándar del sistema, sin aplicar ningún tipo de ordenación adicional sobre los resultados.

A continuación se muestra la implementación de ProdFiltro, que mantiene la misma interfaz pública pero modifica el comportamiento interno.

```

<?php
require_once __DIR__ . '/baseprod.php';

/**
 * Clase ProdFiltro
 * Variante de negocio que aplica una ordenación por precio.
 */
class ProdFiltro extends BaseProd {

    /**
     * Sobrescribe el método buscar() para aplicar una ordenación adicional.
     * En este caso, permite ordenar los resultados por precio
     * en orden ascendente o descendente.
     */
    public function buscar(string $term=''): array {
        // Limpieza del término de búsqueda
        $term = trim($term ?? '');

        // Primero obtenemos los resultados normales del repositorio
        $resultados = $this->repo->buscar($term);

        // Se obtiene el parámetro 'orden' de la URL.
        // Si no existe, por defecto se ordena de mayor a menor.
        $orden = $_GET['orden'] ?? 'desc';

        /**
         * Se ordena el array de resultados por precio.
         * - asc => de menor a mayor
         * - desc => de mayor a menor
         */
        usort($resultados, function($a, $b) use ($orden) {

            $priceA = floatval($a['price'] ?? 0);
            $priceB = floatval($b['price'] ?? 0);

            if ($orden === 'asc') {
                return $priceA <=> $priceB;
            } else {
                return $priceB <=> $priceA;
            }
        });
    }
}

```

```

    }
    });

    // Se devuelven los resultados ya ordenados
    return $resultados;
}
}

```

En la clase `ProdFiltro` se respeta la misma firma del método `buscar(string $term): array`, pero se introduce lógica adicional para ordenar los productos por precio según el parámetro `orden`. Esta diferencia de comportamiento, manteniendo la misma interfaz pública, es lo que permite aplicar el principio de polimorfismo dentro de la capa de negocio.

2.1 Clase base común

Para estructurar la capa de negocio se ha definido una clase abstracta común, `BaseProd`. Esta clase centraliza el repositorio compartido (inyectado por constructor) y establece la firma obligatoria del método `buscar(string $term): array`. De este modo, las clases concretas (`Prod` y `ProdFiltro`) heredan la misma base y quedan obligadas a implementar el método, garantizando una interfaz consistente.

```

<?php
/**
 * Clase base para la capa de negocio.
 * Define la estructura común y obliga a implementar el método buscar().
 */
abstract class BaseProd {
    protected $repo;

    public function __construct($repo) {
        $this->repo = $repo;
    }

    /**
     * Busca productos según la lógica de negocio de la clase concreta.
     */
    abstract public function buscar(string $term=''): array;
}

```

A partir de esta clase base, cada clase concreta implementa `buscar(string $term): array` con su lógica específica.

3. Integración en la interfaz de usuario

La implementación del polimorfismo no se ha limitado únicamente a la capa de negocio, sino que se ha integrado directamente en la interfaz de usuario de la página tienda.php.

Desde la propia interfaz, el usuario puede activar o desactivar el filtro y seleccionar el orden de los productos por precio. Estas acciones modifican los parámetros enviados por GET, concretamente el parámetro *orden*, lo que provoca que la clase ProdFiltro aplique la lógica de ordenación correspondiente.

De este modo, la interfaz no solo muestra información, sino que interactúa directamente con la lógica polimórfica del sistema. El cambio en la URL actualiza el comportamiento interno sin necesidad de modificar la estructura del código cliente, manteniendo desacoplada la presentación de la lógica de negocio.

La selección de ordenación se refleja visualmente en la interfaz mediante un selector desplegable, lo que permite al usuario elegir entre orden ascendente o descendente. Además, el sistema mantiene un comportamiento coherente cuando el parámetro no está presente o es inválido.



Figura 1. Estado sin filtro (orden descendente por defecto)



Figura 2. Activación del filtro y orden ascendente



Figura 3. Orden descendente con filtro activo

Al cambiar el selector a orden descendente, la interfaz actualiza la URL y la petición a la API, provocando una nueva ejecución del método `buscar()` con el parámetro orden correspondiente.

4. Pruebas y validación

Para validar el correcto funcionamiento de la implementación se ha desarrollado un archivo específico de pruebas (`pruebas_ud5.php`). Estas pruebas verifican tanto el comportamiento polimórfico como la correcta aplicación de la ordenación por precio.

En primer lugar, se comprueba que tanto la clase `Prod` como `ProdFiltro` responden al método `buscar(string $term): array` y devuelven un array de resultados. Esta verificación demuestra que ambas clases comparten la misma interfaz pública y pueden utilizarse de forma intercambiable.

Posteriormente, se realizan pruebas específicas sobre la clase `ProdFiltro` con el objetivo de validar la correcta ordenación por precio. Se comprueba que, cuando el parámetro `orden` se establece en `asc`, los productos se devuelven en orden ascendente, y que al utilizar `desc` se ordenan de forma descendente. Asimismo, se verifica que ante un valor inválido el sistema mantiene un comportamiento coherente aplicando el orden descendente por defecto, comportamiento que también se conserva cuando el parámetro no está presente en la petición.

Estas comprobaciones garantizan que la lógica implementada en la capa de negocio responde correctamente a los distintos escenarios posibles.

```
== PRUEBAS UD5 ==

-- Demostración explícita de polimorfismo (misma interfaz, distinto comportamiento) --
[OK] Prod responde a buscar() y devuelve array
[OK] Prod devuelve al menos 1 resultado
[OK] ProdFiltro responde a buscar() y devuelve array
[OK] ProdFiltro devuelve al menos 1 resultado

-- ProdFiltro: pruebas de ordenación por precio (asc/desc) --
[OK] orden=asc devuelve precios en orden ascendente
ASC sample: 24.5, 29.95, 59.9, 139

[OK] orden=desc devuelve precios en orden descendente
DESC sample: 139, 59.9, 29.95, 24.5

[OK] orden inválido se comporta como desc
INVALID sample: 139, 59.9, 29.95, 24.5

[OK] sin orden => por defecto desc
DEFAULT sample: 139, 59.9, 29.95, 24.5
```

Figura 4. Resultado de ejecución de `pruebas_ud5.php`

La ejecución de las pruebas confirma que el sistema responde correctamente en todos los casos evaluados, validando tanto el polimorfismo como la lógica de ordenación.

5. Conclusión

La implementación desarrollada en esta unidad demuestra una aplicación práctica del polimorfismo dentro de un entorno funcional real. No se trata únicamente de crear dos clases con el mismo método, sino de integrarlas en la lógica de negocio de la tienda y conectarlas directamente con la interfaz de usuario.

El uso de las clases *Prod* y *ProdFiltro* permite mantener una misma estructura de acceso a datos mientras se introducen comportamientos distintos según el contexto. La ordenación por precio se gestiona de forma dinámica mediante parámetros GET, lo que demuestra cómo la lógica de negocio puede adaptarse sin modificar el código cliente.

Además, la integración con la interfaz de *tienda.php* evidencia una conexión coherente entre frontend y backend. El usuario puede interactuar con el sistema mediante el selector de ordenación y el estado del filtro, y estos cambios se reflejan correctamente en los resultados mostrados.

Las pruebas realizadas confirman que el comportamiento es consistente tanto en condiciones normales como ante parámetros inválidos, garantizando robustez en la implementación.

En conjunto, esta unidad no solo cumple con los requisitos técnicos de la programación orientada a objetos, sino que integra el concepto de polimorfismo dentro de una funcionalidad completa del proyecto.

6. Recursos

Para la elaboración del proyecto se han utilizado distintos recursos de apoyo:

- *Mimo App y Enki App*
- *Material de la unidad didáctica*
- *Base de código del trabajo entregado para unidad 4.*
- *CodePen*
<https://codepen.io/pen>
- *Program with Gio: Learn PHP The Right Way - Full PHP Tutorial for Beginners & Advanced (Youtube)*
https://youtube.com/playlist?list=PLr3d3QYzkw2xabQRUpCZ_IBk9W50M9pe-&si=Lt00-XbHoDhLyy8i